

Cafe POS System Pitch Proposal

A simple, explainable POS for small cafes that handles real ordering, real payments, MySQL persistence, and one practical AI workflow.

Problem

Small cafes need fast counter tools without the complexity of enterprise POS systems. Staff must create orders, change them mid-meal, handle special requests, split bills, and close payments cleanly while keeping a reliable order history.

Solution

This project provides a focused Angular cashier interface backed by Spring Boot and MySQL. Orders stay open while customers are still at the table. Staff can add items, change quantities, record modifiers, void mistakes, save kitchen notes, preview bill splits, and record partial payments until the order is fully paid.

AI Feature

Gemini is integrated through the backend, not mocked in the frontend. The AI helper can suggest one relevant add-on based on the cart, or turn a complicated order into a concise kitchen summary. The API key is kept server-side through the `GEMINI_API_KEY` environment variable.

Why It Is Practical

Cafe situation	How the system handles it
Customer adds food after drinks	Add item to an open order and totals recalculate.
Customer changes milk or sugar	Edit line modifiers and save kitchen notes.
Wrong item entered	Void the line while preserving history.
Friends split the bill	Preview equal split on the remaining balance.
Multiple payment methods	Record cash, card, or e-wallet payments.

Architecture

Angular calls Spring Boot REST endpoints. Spring Boot owns business rules in `OrderService`, persists through Spring Data JPA, applies Flyway database migrations, and calls Gemini using a server-side HTTP client. MySQL stores menu items, orders, order lines, and payments.

Submission Value

The code is intentionally compact and straightforward: controllers are thin, DTOs define API shapes, entities map to the database, and one service contains the billing rules. This makes the project easy to demonstrate and easy to explain.